



Laboratory Report

LECTURER	John ORaw
STUDENT NAME	Liam Saunders
STUDENT NUMBER	L00187849
PROGRAMME	MSc Cloud
MODULE CODE	INFS_IT903
ASSIGNMENT TITLE	Academic Report - PowerShell
SUBMISSION DATE	

Student Declaration

1. I have accurately identified and included the sources of all facts, ideas, opinions, and viewpoints from others in the assignment references. All direct quotations, paraphrasing, and discussions of ideas from books, journal articles, internet sources, course materials, or any other sources used are properly acknowledged and cited in the assignment references.
2. I have not used unauthorised artificial intelligence tools or aids.
3. I understand and am compliant with ATU's policy and procedures regarding Academic Integrity and I am aware of the consequences of any violations.
4. I have followed the referencing guidelines recommended in the assignment instructions and / or programme documentation.
5. By signing this form or by submitting material online I confirm that this assignment, or any part of it, has not been previously submitted by me or any other person for assessment on this or any other programme of study.
6. By signing this form or by submitting material for assessment online I confirm that I have read and understood [AQAE022 Academic Integrity Policy](#)

Student Signature	Date
Liam Saunders	10/12/2025

Contents

Description.....	4
Aims.....	4
Method	5
Results and Testing.....	6
Conclusions	7
References	8
Appendices.....	9
Appendix A – Scripts.....	9
Appendix B – Script Outputs	19

Figure 1: Setup.ps1 - Initial PowerShell environment configuration	9
Figure 2: Install-PowerShell.ps1 - This is an official Microsoft installer script and is not reproduced here due to length.....	9
Figure 3: Download PowerShell7.ps1 - Downloads official installer script	10
Figure 4: Install PowerShell7.ps1 - Automated installation of PowerShell 7	10
Figure 5: Verify PowerShell7.ps1 - Checks module paths and version.....	10
Figure 6: FindModules.ps1 - Lists all PowerShell Gallery modules	11
Figure 7: FindModulesPSCore.ps1 - Finds modules supporting PowerShell Core	11
Figure 8: Creating a Module.ps1 - Creates a custom HelloWorld module	12
Figure 9: Variables.ps1 - Demonstrates variable behaviour and types	13
Figure 10: Tax Calculation Script.ps - Example calculates the VAT and gross amount	14
Figure 11: Types.ps1 - Shows casting and numeric types.....	14
Figure 12: if and elseif.ps1 - Conditional branching demonstration	15
Figure 13: Switch.ps1 - Outputs values using switch statement.....	16
Figure 14: For Loop.ps1 - Counts values in a for loop	17
Figure 15: For Each.ps1 - Iterates through an array	17
Figure 16: While Loop.ps1 - Runs repeated logic until input changes	18
Figure 17: Do Until.ps1 - Loops until numeric condition is met.....	18
Figure 18: Piping Commands.ps1 - Demonstrates the PowerShell pipeline.....	19
Figure 19 Setup.ps1 Output - Displays current PowerShell version.....	19
Figure 20: Install-PowerShell.ps1 Output	19
Figure 21: Download PowerShell7.ps1 Output - installer script downloaded to directory.....	20
Figure 22: Install PowerShell7.ps1 Output - PowerShell 7 installation initiated successfully	20
Figure 23: Verify PowerShell7.ps1 Output - Lists all active module paths	20
Figure 24: PowerShell Extension installation	21
Figure 25: FindModules.ps1 Output - Shows total number of Gallery modules	21
Figure 26: FindModulesPSCore.ps1 Output.....	22
Figure 27: Creating a Module.ps1 Output	22
Figure 28: .psm1 file creation in Documents/PowerShell/Modules/HelloWorld Directory	23
Figure 29: Variables.ps1 Output - Displays variable contents and type behaviour	23
Figure 30: Types.ps1 Output	23
Figure 31: if and elseif.ps1 Output	24
Figure 32: Switch.ps1 Output.....	24
Figure 33: For Loop.ps1 Output - Displays incrementing numeric values.....	25
Figure 34: For Each.ps1 Output - Prints each letter in sequence.....	25
Figure 35: While Loop.ps1 Output - Repeated messages until stop condition.....	25
Figure 36: Do Until.ps1 Output - Shows increasing values until limit reached.....	26
Figure 37: Piping Commands.ps1 Output - Formatted directory listing displayed	26

Description

The laboratory report documents the development, installation and evaluation of a PowerShell 7 scripting environment used for automation, alongside the building and development of other scripts designed to show the features and benefits of using PowerShell as a viable programming tool. The work carried out on a virtual machine running windows 11 with a dedicated working directory, ensuring all scripts could be stored, managed and reused on a consistent basis. The first setup script was created to check the existing PowerShell version, configure the execution policy, install the NUGet package provider, update PowerShellGet and create a working PowerShell directory. The second script focused primarily on downloading Microsoft's official PowerShell installation script directly from Microsoft's secure endpoint. Running this script confirmed successful retrieval with no errors occurring during execution. The Installation was then automated using a third script in a more structured manner.

The laboratory then progressed to using Visual studio code to run, execute and debug scripts by installing the PowerShell extension. Dedicated scripts were created to demonstrate all module's available in the PowerShell gallery and to search for those tagged as compatible with PowerShell Core. Further work focused on core scripting fundamentals using PowerShell in a sequence of scripts examining variables, type handling, and arithmetic operations, including VAT calculation script. Control-flow logic was demonstrated and learned through branching structures, switch statements. Iterative processing was demonstrated using loops, foreach loops, while loops, and do-until loops. The final script demonstrated the pipeline behaviour by formatting the current working directory output and directing it to the host. These scripts were completed and functioned using PowerShell 7 automation, providing a strong foundation for future work within the module Infrastructure-as-Code.

Aims

The aims of this laboratory were to develop and understand the use of PowerShell 7 as a scripting environment. The work involved installing, configuring and exploring module behaviour within PowerShell to create a series of customizable scripts to explore PowerShell concepts. The aims of this lab are outlined as follows:

1. To perform the installation of PowerShell 7 and to configure it using scripted methods that prepare use for an execution environment.
2. To learn the behaviour of PowerShell modules, investigate PowerShell scripting concepts, including variables, data types, type conversion and arithmetic operations, through the execution of dedicated scripts.
3. To implement a range of control-flow operations, switch statements and different types of loop constructs, to get a better understanding of how PowerShell manages these logical decision-making aspects.
4. To explore PowerShell's pipeline by developing scripts that test the flow and formatting and presentation of objects as they pass through different commands.
5. To create a well put together, repeatable workflow for developing PowerShell scripts, forming a good foundation for future automation within this module.

Method

The laboratory work was carried out within a Windows 11 Virtual machine using VM Workstation, providing a stable and standardised environment suitable for installation and configuring and testing of PowerShell 7. The use of both PowerShell ISE and Visual Studio Code was used to run, execute and debug all scripts. PowerShell Core represents the modern evolution of the Windows automation ecosystem, offering cross-platform compatibility, object-oriented processing and deep integration with system administration tools across Windows, Linux and macOS(Microsoft 2023a).

The configuration of the PowerShell 7 environment started with the execution of the *Setup.ps1* script, shown in Figure 1, which established the pre-requisites needed for the installation of PowerShell 7. The script executed enabled the NuGet package provider and ensured that the PowerShellGet module was installed with the latest version. This created a working directory at C:\PowerShell, which served as the repository for all scripts outlined in the report. This pre-configuring steps align with Microsoft's recommended installation workflow and provided the foundations for future automation(Microsoft 2023a).

Following this, an official Microsoft installation script (*install-PowerShell.ps1*) as shown in Figure 2. This script is not reproduced in the appendix fully due to its length but was executed as part of the installation workflow and can be access from Microsoft official documentation(Microsoft 2023a). The next step was to download the official bootstrapper through REST-based automation using the script presented in Figure 3(*Download PowerShell7.ps1*). This script retrieved the installer using the (*Invoke-RestMethod*) and saved it locally. This aligns with the modern DevOps practices in which scripts are pulled dynamically from trusted endpoints rather than being transferred manually (Microsoft 2023b). The installation of PowerShell 7 shown in Figure 4, the use of parameters such as (*Quiet*, *UseMSI* and *EnablePSRemoting*) are best practice for ensuring the script can be reproduced during deployment (Maurer 2019). Verification of PowerShell 7 using script shown in Figure 5 which verified the module directories available to the newly install PowerShell 7 environment.

As shown in Figure 6, module exploration was done by executing the (*Find-Module*) to list all available modules within PowerShell Gallery. The next script shown in Figure 7 filtered the modules tagged with PowerShell Core. In Figure 8 a custom module was created through the script which constructed a directory structure and generated a (*.psm1*) module file. Verification was successful using the (*Get-Module -List Available*) syntax which confirmed that the module was structured and discovered by PowerShell(Hicks 2022a).

Scripting concepts were explored next. The script in Figure 9 showed variable behaviour, including clearing, deletion and type inference. String interpolation and Arithmetic was established in Figure 10, which implemented a VAT calculation program. This task reinforced the use of variables, operators and formatting outputs. A more detailed examination of this was carried out in Figure 11(*Types.ps1*), which shows dynamic typing, PowerShell's integration with .NET types (Hicks 2022b).

Conditional logic was then demonstrated using the (*if/elseif*) syntax shown in Figure 12, followed by the (*switch*) statement in Figure 13. These are control-flow structures which has been tested using numerical values and string patterns. PowerShell's ability to view expressions and match them provide a powerful means of simplifying complex logic problems.

The next section of the laboratory involved loop constructs. *For Loop.ps1* (Figure 14) demonstrated fixed-iteration loops, *For Each.ps1* (Figure 15) shows a collection-based iteration. *While Loop.ps1* (Figure 16) explores controlled looping, while the *Do Until.ps1* script in Figure 17 validate loop execution that continues until a termination command or condition is met. These constructs provided a complete understanding of iterative logic using loops withing PowerShell. (Hicks and Jones 2020)

The final and remaining script in the laboratory was demonstrated in Figure 18, which examines PowerShell's object pipeline though the syntax (*DIR | Format-Table | Out-Host*). This demonstrates object flow rather than text flow which highlights a core advantage of PowerShell over other shells, allowing for more reliable and structured automation pipelines (Hansen and Hicks 2020).

Results and Testing

To validate the fun actionality of all scripts, each file underwent execution testing across two standard PowerShell development environments, PowerShell ISE and Visual Studio Code. All scripts were executed without errors, handled input correctly and behaved consistently across each editor.

The first script produced the configuration output shown in Figure 19, confirming that the execution policy was updated and that the PowerShellGet and the NuGet provider were active. The Microsoft installer script, reference in Figure 20, downloaded and installed as expected, followed by a successful retrieval of PowerShell 7 (Figure 21). The installation script itself in Figure 22 generated the confirmation displayed in Figure 23 verifying that PowerShell 7 had been installed and registered within the Virtual Machine.

The next scripts were all executed within Visual studio Code by installing the PowerShell extension within Visual Studio Code see (Figure 24). Verification results are shown in Figure 25 confirmed that the module paths were correctly populated and aligned with PowerShell 7. The next script output revealed many modules available for use while tagging the (*PSEdition_Core*) shown in Figure 26. The custom module that was created during the laboratory was successfully recognised, as illustrated in Figure 27 creating the module (*.psm1*) file in the directory shown in Figure 28 as expected. Variable and type demonstrations produced the expended results in (Figure 29, Figure 30).

Branching outputs in (Figure 31, Figure 32) produced the expected behaviour of both *if/elseif* and *Switch* constructs. The Loop results shown in (Figure 33, Figure 34, Figure 35, Figure 36) confirmed the correct iteration patterns across fixed, while-based and conditional loop types. The pipeline demonstration in Figure 37 produced a correctly formatted directory listing, validating object flow through the pipeline.

Conclusions

The main aims of the PowerShell 7 Laboratory were achieved comprehensively, with each stage of the work validating the outputs executed across all scripts. The installation and configuration throughout the laboratory provided the successful automation of environment preparation and module retrieval, confirming that PowerShell 7 has been deployed and installed correctly within the Windows 11 virtual machine. The scripts following this, including module searching, variable control, branching logic, looping structures and pipeline operations, returned results on a consistent basis. Providing evidence of a clear understanding of how PowerShell processes objects, executes logic and interacts with system resources. With all this in mind, the results confirm that the progression from environment setup to advanced scripting mirrors modern automation practices that emphasise reproducibility. (Microsoft 2023c).

Positioning this work within the current developments in cloud computing highlights its wider relevance. PowerShell Core is specifically designed as a cross-platform automation tool capable of supporting most modern hybrid cloud environments (Microsoft 2025). The scripting exercises within this laboratory reflect the same foundational concepts that reflect in professional configuration automation. The investigation into module searching and compatibility, for example, aligns with new practices in modular automation, where companies depend on curated, version-controlled libraries to support scalable deployment patterns (Microsoft 2023d). The iterative constructs throughout this laboratory, involving loops, switches, and conditionals, reflect the same behaviours implemented at scale in systems and cloud environments, situating this work firmly within contemporary automation and operational research (Kim *et al.* 2016).

Critical awareness of the outcomes of these scripts reveals opportunities for refinement and improvements. While automated installs are convenient, we can enhance their security by implementing more robust exception handling, logging, and signature validation. Module discovery scripts return limited results; filtering by publisher or version would focus on clearer outcomes. Control-flow scripts worked as expected but could evolve toward parameter-driven functionality to support more dynamic runtime behaviours. Implementing more sophisticated pipeline operations, enhancing output formatting, and conducting thorough testing could lead to a more resilient automation pattern. By adopting these improvements, we can ensure a more secure and efficient automation process. Furthermore, incorporating user feedback into the development cycle will allow us to continually adapt and optimise these scripts for better performance and usability. Introducing automated verification routines, rather than relying solely on manual inspection, would enhance reliability and align with contemporary industry practices in continuous testing and configuration drift detection. (Humble and Farley 2010).

In conclusion, this laboratory experience transcends the mere execution of discrete PowerShell scripts and embodies the broader development of automation literacy, which is crucial for cloud engineering and Infrastructure as Code. Each script not only highlights technical proficiency but also illustrates the ability to contextualise procedural logic within the wider discourse of configuration automation, modular tooling, and operational scalability. Throughout this laboratory the insights gained advocate for a deeper integration of PowerShell within future automation projects. Collectively, the laboratory contributes to the emerging capability of constructing dependable, scalable, and automated cloud infrastructure solutions grounded in contemporary PowerShell 7 practice.

References

- Hansen, B. and Hicks, J. (2020) *Learn PowerShell in a Month of Lunches*, New York: Manning Publications.
- Hicks, J. (2022a) Exploring PowerShell Modules with Get-Module [online], available: <https://jdhitsolutions.com/blog/powershell/>.
- Hicks, J. (2022b) Understanding the PowerShell Type System [online], available: <https://jdhitsolutions.com/blog/powershell/>.
- Hicks, J. and Jones, D. (2020) *PowerShell Scripting and Toolmaking*, New York: Manning Publications.
- Humble, J. and Farley, D. (2010) *Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation*, Boston: Addison-Wesley Professional.
- Kim, G., Humble, J., Debois, P., and Willis, J. (2016) *The DevOps Handbook: How to Create World-Class Agility, Reliability, and Security in Technology Organizations*, Portland, Oregon: IT Revolution Press.
- Maurer, T. (2019) How to Install and Update PowerShell 7 [online], available: <https://www.thomasmaurer.ch/2019/07/how-to-install-and-update-powershell-7/>.
- Microsoft (2023a) PowerShell Overview [online], available: <https://learn.microsoft.com/powershell/scripting/overview> [accessed 10 Dec 2025].
- Microsoft (2023b) Invoke-RestMethod (PowerShell Documentation) [online], available: <https://learn.microsoft.com/en-us/powershell/module/microsoft.powershell.utility/invoke-restmethod>.
- Microsoft (2023c) PowerShell Scripting Overview [online], available: <https://learn.microsoft.com/en-us/powershell/scripting/overview>.
- Microsoft (2023d) Authoring and Managing PowerShell Modules [online], available: <https://learn.microsoft.com/en-us/powershell/scripting/developer/module/authoring>.
- Microsoft (2025) What Is PowerShell? [online], available: <https://learn.microsoft.com/en-us/powershell/scripting/overview?view=powershell-7.5>.

Appendices

Appendix A – Scripts

```
<#
Name: Liam Saunders
Student Number: L00187849
Date: 05 Dec 2024
Task: Pre-requisite for upgrading to PowerShell 7
#>

# Check the existing PowerShell version
$PSVersionTable

# Set an execution policy
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Force

# Install NuGet as a package provider
Install-PackageProvider NuGet -MinimumVersion 2.8.5.201 -Force | Out-Null

# Install the PowerShellGet module
Install-Module -Name PowerShellGet -Force -AllowClobber

# Create a script directory
mkdir C:\PowerShell
```

Figure 1: Setup.ps1 - Initial PowerShell environment configuration

```
# Copyright (c) Microsoft Corporation.
# Licensed under the MIT License.
<#
.Synopsis
    Install PowerShell on Windows, Linux or macOS.
.DESCRIPTION
    By default, the latest PowerShell release package will be installed.
    If '-Daily' is specified, then the latest PowerShell daily package will be installed.
.Parameter Destination
    The destination path to install PowerShell to.
.Parameter Daily
    Install PowerShell from the daily build.
    Note that the 'PackageManagement' module is required to install a daily package.
.Parameter DoNotOverwrite
    Do not overwrite the destination folder if it already exists.
.Parameter AddToPath
    On Windows, add the absolute destination path to the 'User' scope environment variable 'Path';
    On Linux, make the symlink '/usr/bin/pwsh' points to "$Destination/pwsh";
    On MacOS, make the symlink '/usr/local/bin/pwsh' points to "$Destination/pwsh".
EXAMPLE
```

Figure 2: Install-PowerShell.ps1 - This is an official Microsoft installer script and is not reproduced here due to length.

```
<#
Name: Liam Saunders
Student Number: L00187849
Date: 05 Dec 2024
Task: Download the PowerShell 7 installation script
#>

# Download PowerShell 7 installation script
Set-Location C:\PowerShell
$URI = "https://aka.ms/install-powershell.ps1"
Invoke-RestMethod -Uri $URI |
Out-File -FilePath C:\PowerShell\Install-PowerShell.ps1
```

Figure 3: Download PowerShell7.ps1 - Downloads official installer script

```
|<#
Name: Liam Saunders
Student Number: L00187849
Date: 05 Dec 2024
Task: A script to install PowerShell 7
#>

$MYPARAMS = @{
    UseMSI                = $true
    Quiet                  = $true
    AddExplorerContextMenu = $true
    EnablePSRemoting       = $true
}

C:\PowerShell\Install-PowerShell.ps1 @MYPARAMS
```

Figure 4: Install PowerShell7.ps1 - Automated installation of PowerShell 7

```
$I = 0
$env:PSModulePath -split ';' |
Foreach-Object {"[{0:N0}] {1}" -f $I++, $_}
```

Figure 5: Verify PowerShell7.ps1 - Checks module paths and version

Laboratory Report

```
<#
Name: Liam Saunders
Student Number: L00187849
Date: 05 Dec 2024
Task: A script to find modules in PowerShell
Note: This command may take some time to run depending on network and repository response.
#>

$PGSM = Find-Module -Name *
"There are {0:N0} modules in the PowerShell Gallery." -f $PGSM.Count
|
```

Figure 6: FindModules.ps1 - Lists all PowerShell Gallery modules

```
<#
Name: Liam Saunders
Student Number: L00187849
Date: 05 Dec 2024
Task: A script to find modules which support PowerShell Core. Includes an additional command to display the total number of modules.
Note: This command may take some time to run.
#>

# Find modules that explicitly support PowerShell Core
$PGSM = Find-Module -Name * -Tag 'PSEdition_Core'
"There are {0:N0} modules that support PowerShell Core." -f $PGSM.Count

# Optional command to list all available modules
# Get-Module -ListAvailable
```

Figure 7: FindModulesPSCore.ps1 - Finds modules supporting PowerShell Core

```
<#
Name: Liam Saunders
Student Number: L00187849
Date: 05 Dec 2024
Task: A script to create a custom PowerShell module and verify availability.
Note: Demonstrates module folder structure, function definition, and module discovery.
#>

# Define the module path within the user's PowerShell module directory
$MyModulePath = "C:\Users\$env:USERNAME\Documents\PowerShell\Modules\HelloWorld"

# Define the content of the module using a here-string
$MyModule = @"
# HelloWorld.psm1
Function Get-HelloWorld {
    "Hello World from Liam Saunders"
}
"@

# Create the directory if it does not already exist
New-Item -Path $MyModulePath -ItemType Directory -Force | Out-Null

# Output the module file into the correct location
$MyModule | Out-File -FilePath "$MyModulePath\HelloWorld.psm1"

# Verify that the module is available on the system
Get-Module -Name HelloWorld -ListAvailable
|
```

Figure 8: Creating a Module.ps1 - Creates a custom HelloWorld module

```
<#
Name: Liam Saunders
Student Number: L00187849
Date: 08 Dec 2025
Task: Using variables in PowerShell
#>

# Displays the list of available variables
Get-Variable

# Declaring a variable containing mixed object types (array, string, symbols)
$Mixed_Variable = 1, 2, "a", "ff"
$Mixed_Variable

# Clearing a variable value
Clear-Variable -Name Mixed_Variable
$Mixed_Variable

# Removing the variable completely
Remove-Variable -Name Mixed_Variable

# Re-declaring the variable to demonstrate object typing
$Mixed_Variable = 1, 2, "a", "ff"
$Mixed_Variable.GetType()

# Demonstrating a strongly typed integer variable
[int]$Mixed_Variable = 1
$Mixed_Variable

# Showing automatic type conversion when assigning a numeric string
[int]$Mixed_Variable = 1
$Mixed_Variable = "123456789"
$Mixed_Variable.GetType()

# Showing that PowerShell cannot convert alphabetic strings into integers
[int]$Mixed_Variable = 1
$Mixed_Variable = "This will give you an error!"
$Mixed_Variable

# Translating a date string into a DateTime object
[datetime]$TodayDate = "12/08/2025"
$TodayDate

# Demonstrating creation of a new read-only variable with a description
Remove-Variable LiamVariable -ErrorAction SilentlyContinue
New-Variable LiamVariable -Value 3.142 -Description "PI with write-protection" -Option ReadOnly
Get-Variable LiamVariable
```

Figure 9: Variables.ps1 - Demonstrates variable behaviour and types

```
<#
Name: Liam Saunders
Student Number: L00187849
Date: 08 Dec 2025
Task: A script to calculate VAT and produce a formatted output message.
#>

# Declare the net value and VAT rate
$NET = 111
$VAT = 0.23

# Calculate VAT amount based on the net value
$VATAmount = $NET * $VAT

# Calculate the gross amount
$Gross = $NET + $VATAmount

# Create a formatted text output
$text = "The total €$Gross is the sum of the net value €$NET with the VAT amount €$VATAmount at $($VAT * 100)% VAT rate."

# Display the text
$text
|
```

Figure 10: Tax Calculation Script.ps - Example calculates the VAT and gross amount

```
<#
Name: Liam Saunders
Student Number: L00187849
Date: 08 Dec 2025
Task: Introduction to Types
#>

# Demonstrating string behaviour
$stringValue = "Hello Testing Testing!"
$stringValue.ToUpper()
$stringValue.ToLower()

# Demonstrating array types in PowerShell
$MyArray = 1, 2, 3, 4, 5
$MyArray[1] # Shows the second element (index begins at 0)

# Example using implicit integer typing
$LittleNumber = 12345
$LittleNumber.GetType()

# Example using a large integer that PowerShell automatically treats as Int64
$BigNumber = 123456789123456789
$BigNumber.GetType()

# Demonstrating floating-point types
[float]$Float32 = 12.12
$Float32.GetType()

[double]$Float64 = 12345.1234
$Float64.GetType()
```

Figure 11: Types.ps1 - Shows casting and numeric types

```
<#
Name: Liam Saunders
Student Number: L00187849
Date: 08 Dec 2025
Task: Introduction to if and elseif
Description:
Conditional branching
#>

# Basic IF example
$ValueA = 32
$ValueB = 32

if ($ValueA -ne $ValueB) {
    Write-Output "The condition was true"
}

# -----
# IF / ELSEIF example using days of the week
$DayNumber = 3

if ($DayNumber -eq 0) { $DayName = 'Sunday' }
elseif ($DayNumber -eq 1) { $DayName = 'Monday' }
elseif ($DayNumber -eq 2) { $DayName = 'Tuesday' }
elseif ($DayNumber -eq 3) { $DayName = 'Wednesday' }
elseif ($DayNumber -eq 4) { $DayName = 'Thursday' }
elseif ($DayNumber -eq 5) { $DayName = 'Friday' }
elseif ($DayNumber -eq 6) { $DayName = 'Saturday' }

$DayName

# -----
# Using -like to match patterns
$MyName = "Liam"

if ($MyName -like "Lia*") {
    Write-Output "That's my nickname"
}

# -----
# Using user input with multiple conditional checks
<#
This example accepts user input, checks whether it matches the first letters
of the author's name or the lecturer's initials, and outputs an appropriate
response.
#>

$UserInput = Read-Host "Please enter the creator's name: "

if ($UserInput -like "Lia*") {
    Write-Output "That's the Student!"
}
elseif ($UserInput -like "JOR*") {
    Write-Output "That's the lecturer!"
}
```

Figure 12: *if and elseif.ps1* - Conditional branching demonstration

```
<#
Name: Liam Saunders
Student Number: L00187849
Date: 08 Dec 2025
Task: Introduction to Switch
Description:
This script demonstrates how the PowerShell switch statement evaluates a
value and executes a matching block of instructions. Switch branching
is commonly used when comparing one variable against multiple possible values.
#>

# -----
# Switch statement for day of the week
$DayNumber = 1

switch ($DayNumber) {
    0 { $DayName = 'Sunday' }
    1 { $DayName = 'Monday' }
    2 { $DayName = 'Tuesday' }
    3 { $DayName = 'Wednesday' }
    4 { $DayName = 'Thursday' }
    5 { $DayName = 'Friday' }
    6 { $DayName = 'Saturday' }
}

$DayName

# -----
# Switch statement for month of the year
$MonthNumber = 12

switch ($MonthNumber) {
    1 { $MonthName = 'January' }
    2 { $MonthName = 'February' }
    3 { $MonthName = 'March' }
    4 { $MonthName = 'April' }
    5 { $MonthName = 'May' }
    6 { $MonthName = 'June' }
    7 { $MonthName = 'July' }
    8 { $MonthName = 'August' }
    9 { $MonthName = 'September' }
    10 { $MonthName = 'October' }
    11 { $MonthName = 'November' }
    12 { $MonthName = 'December' }
}

$MonthName
```

Figure 13: Switch.ps1 - Outputs values using switch statement

```
<#
Name: Liam Saunders
Student Number: L00187849
Date: 08 Dec 2025
Task: For loop counting script
Description:
#>

# Initialising counter, checking condition, and incrementing by 1 each loop.
for ($counter = 0; $counter -lt 10; $counter++)
{
    $counter
}
```

Figure 14: For Loop.ps1 - Counts values in a for loop

```
<#
Name: Liam Saunders
Student Number: L00187849
Date: 08 Dec 2025
Task: Introduction to Foreach loops
Description:
This script demonstrates how a foreach loop iterates through each element
of an array and processes the values one at a time.
#>

# Creating an array of characters
$CharacterArray = "L", "i", "a", "m"

# Looping through each element in the array
foreach ($Letter in $CharacterArray)
{
    $Letter
}
```

Figure 15: For Each.ps1 - Iterates through an array

```
<#
Name: Liam Saunders
Student Number: L00187849
Date: 08 Dec 2025
Task: Introduction to While loops
#>

# -----
# Example 1 - Simple While loop with numeric increment
# -----

# Initialise the variable before evaluating it in the loop
$value = 0

while ($value -ne 5)
{
    $value++
    Write-Host "The value is now: $value"
}

# -----
# Example 2 - While loop combined with user input and switch branching
# -----

<#
This loop reads user input until the value "P" is entered.
Each recognised letter triggers a specific output message.
#>

while (($InputValue = Read-Host -Prompt "Select a command to run (H, A, R or P):") -ne "P")
{
    switch ($InputValue)
    {
        H { Write-Output "Help option selected." }
        A { Write-Output "Action option selected." }
        R { Write-Output "Reset option detected." }
        P { Write-Output "Process termination requested." }
        default { Write-Output "Input not valid. Acceptable values are H, A, R, or P." }
    }
}
```

Figure 16: While Loop.ps1 - Runs repeated logic until input changes

```
<#
Name: Liam Saunders
Student Number: L00187849
Date: 08 Dec 2025
Task: Creating a Do-Until loop in PowerShell
#>

# Initialise variable
$value = 2

do {
    Write-Output "Starting loop with the number $value"
    Write-Output $value

    # Increment the variable by 2
    $value += 2

    Write-Output "Now the value is $value"
} until ($value -ge 20)
```

Figure 17: Do Until.ps1 - Loops until numeric condition is met

```
<#
Name: Liam Saunders
Student Number: L00187849
Date: 08 Dec 2025
Task: Creating a piped command in PowerShell
Description:
This script demonstrates how pipeline operators allow the output of one
command to become the input of the next.#>

# Piping directory contents into a formatted table and sending output to the host
Dir | Format-Table | Out-Host
```

Figure 18: Piping Commands.ps1 - Demonstrates the PowerShell pipeline

Appendix B – Script Outputs

```
PS C:\Windows\system32> C:\PowerShell\1. Setup.ps1

Name                Value
----                -
PSVersion            5.1.22621.4249
PSEdition            Desktop
PSCompatibleVersions {1.0, 2.0, 3.0, 4.0...}
BuildVersion         10.0.22621.4249
CLRVersion           4.0.30319.42000
WSManStackVersion    3.0
PSRemotingProtocolVersion 2.3
SerializationVersion 1.1.0.1
```

Figure 19 Setup.ps1 Output - Displays current PowerShell version

```
PS C:\Windows\system32> C:\PowerShell\Install-PowerShell.ps1
VERBOSE: Destination: C:\Users\Liam\AppData\Local\Microsoft\powershell
VERBOSE: About to download package from 'https://github.com/PowerShell/PowerShell/releases/download/v7.5.4/PowerShell-7.5.4-win-x64.zip'
VERBOSE: Removing old installation: C:\Users\Liam\AppData\Local\Microsoft\powershell
PowerShell has been installed at C:\Users\Liam\AppData\Local\Microsoft\powershell
PS C:\Windows\system32>
```

Figure 20: Install-PowerShell.ps1 Output

Laboratory Report

```
LastAccessTimeUtc : 08/12/2025 10:54:36
LastWriteTime     : 08/12/2025 10:54:36
LastWriteTimeUtc  : 08/12/2025 10:54:36
Attributes        : Directory
Mode              : d-----
BaseName          : PowerShell
Target            : {}
LinkType          :

PS C:\Windows\system32> <#
Name: Liam Saunders
Student Number: L00187849
Date: 05 Dec 2024
Task: Download the PowerShell 7 installation script
#>

# Download PowerShell 7 installation script
Set-Location C:\PowerShell
$URI = "https://aka.ms/install-powershell.ps1"
Invoke-RestMethod -Uri $URI |
Out-File -FilePath C:\PowerShell\Install-PowerShell.ps1

PS C:\PowerShell> Get-Help -Name C:\PowerShell\Install-PowerShell.ps1
Install-PowerShell.ps1 [-Destination <string>] [-Daily] [-DoNotOverwrite] [-AddToPath] [-Preview] [<CommonParameters>]
Install-PowerShell.ps1 [-UseMSI] [-Quiet] [-AddExplorerContextMenu] [-EnablePSRemoting] [-Preview] [<CommonParameters>]

PS C:\PowerShell>
```

Figure 21: Download PowerShell7.ps1 Output - installer script downloaded to directory

```
PS C:\PowerShell> <#
Name: Liam Saunders
Student Number: L00187849
Date: 05 Dec 2024
Task: A script to install PowerShell 7
#>

$MYPARAMS = @{
    UseMSI           = $true
    Quiet            = $true
    AddExplorerContextMenu = $true
    EnablePSRemoting = $true
}

C:\PowerShell\Install-PowerShell.ps1 @MYPARAMS

VERBOSE: About to download package from 'https://github.com/PowerShell/PowerShell/releases/download/v7.5.4/PowerShell-7.5.4-win-x64.msi'

PS C:\PowerShell>
```

Figure 22: Install PowerShell7.ps1 Output - PowerShell 7 installation initiated successfully

```
VERBOSE: About to download package from 'https://github.com/PowerShell/PowerShell/releases/download/v7.5.4/PowerShell-7.5.4-win-x64.msi'

PS C:\PowerShell> $I = 0
$env:ModulePath -split ';' |
Foreach-Object {"[{0:N0}] {1}" -f $I++, $_}
[0] C:\Users\Liam\Documents\WindowsPowerShell\Modules
[1] C:\Program Files\WindowsPowerShell\Modules
[2] C:\Windows\system32\WindowsPowerShell\v1.0\Modules

PS C:\PowerShell> |
```

Figure 23: Verify PowerShell7.ps1 Output - Lists all active module paths

Laboratory Report

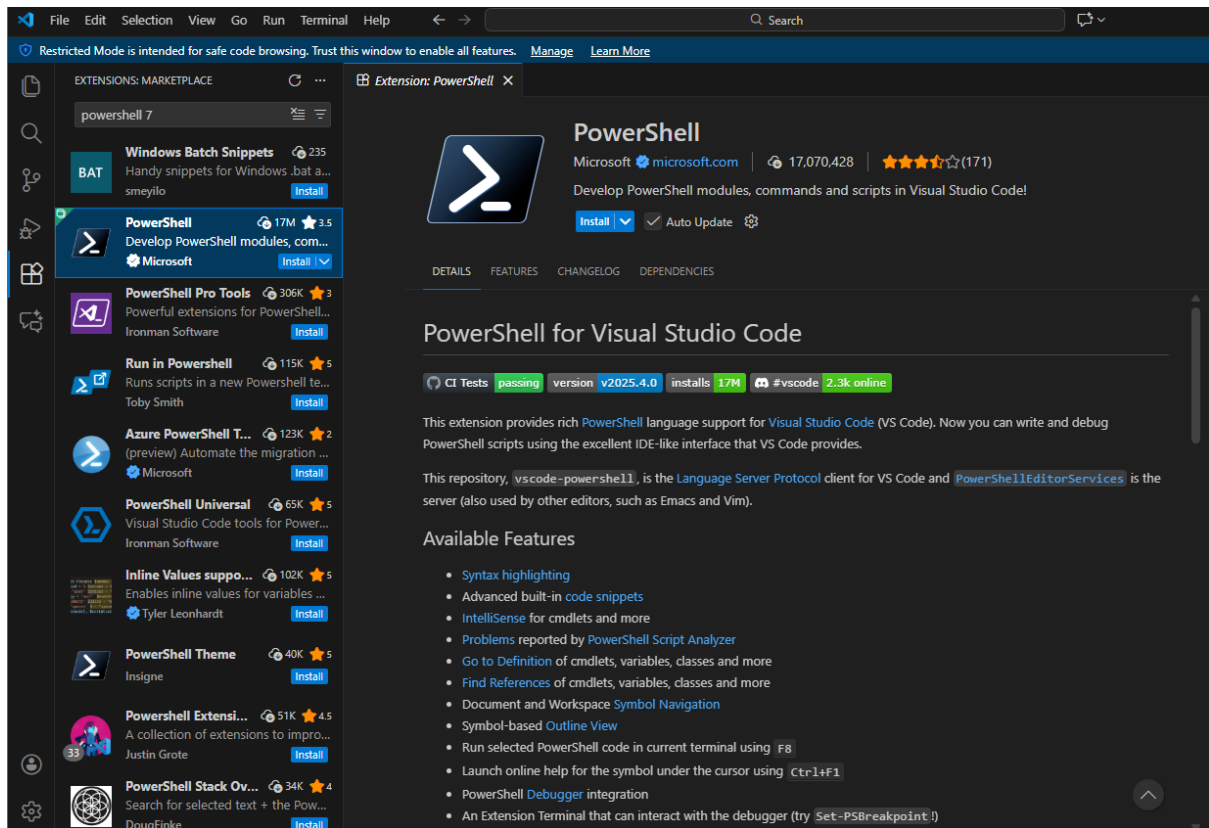


Figure 24: PowerShell Extension installation

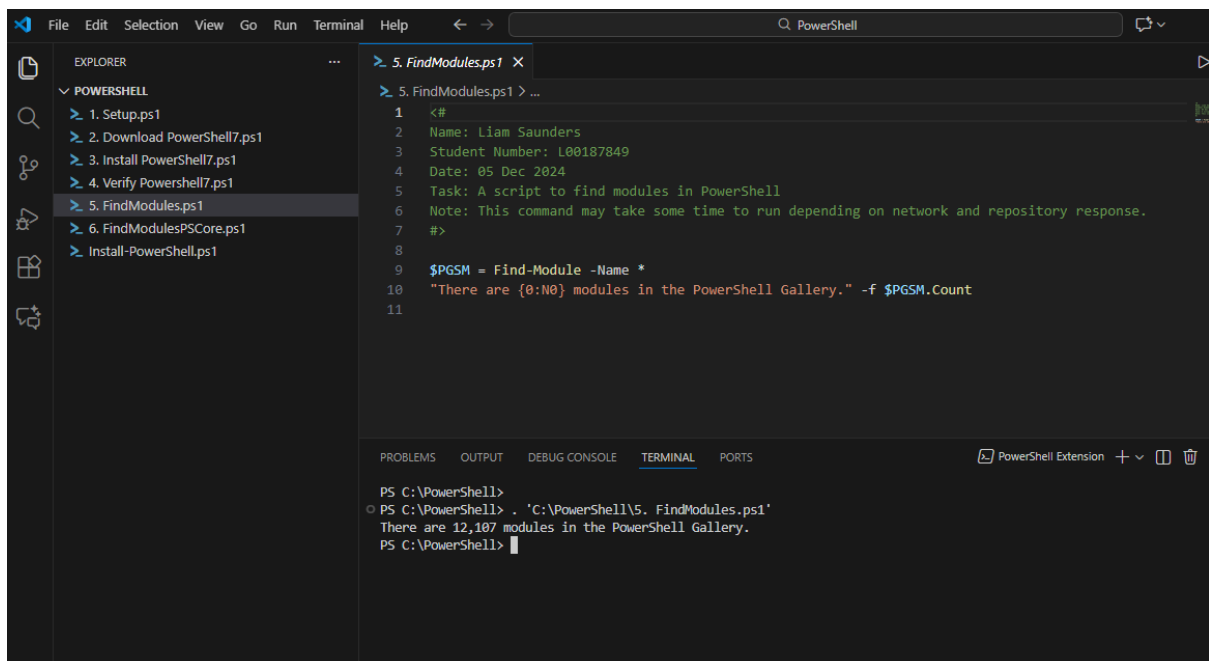
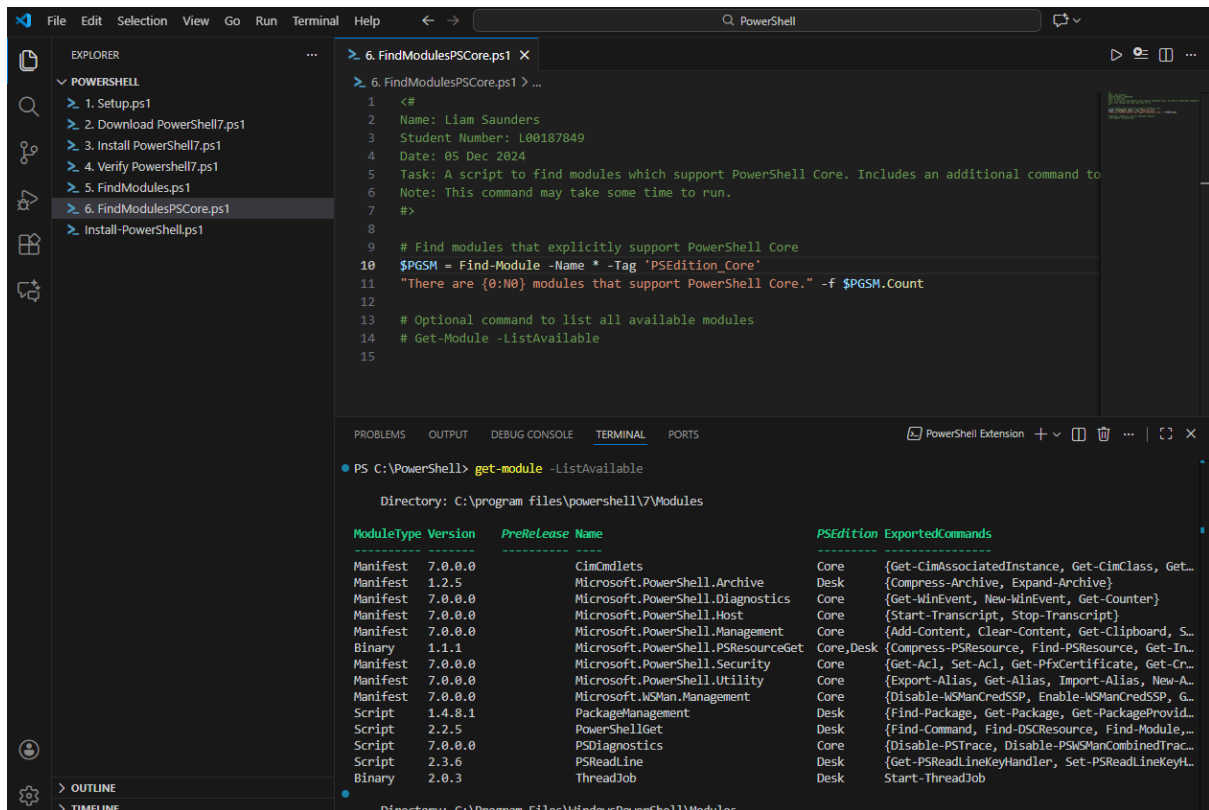


Figure 25: FindModules.ps1 Output - Shows total number of Gallery modules

Laboratory Report



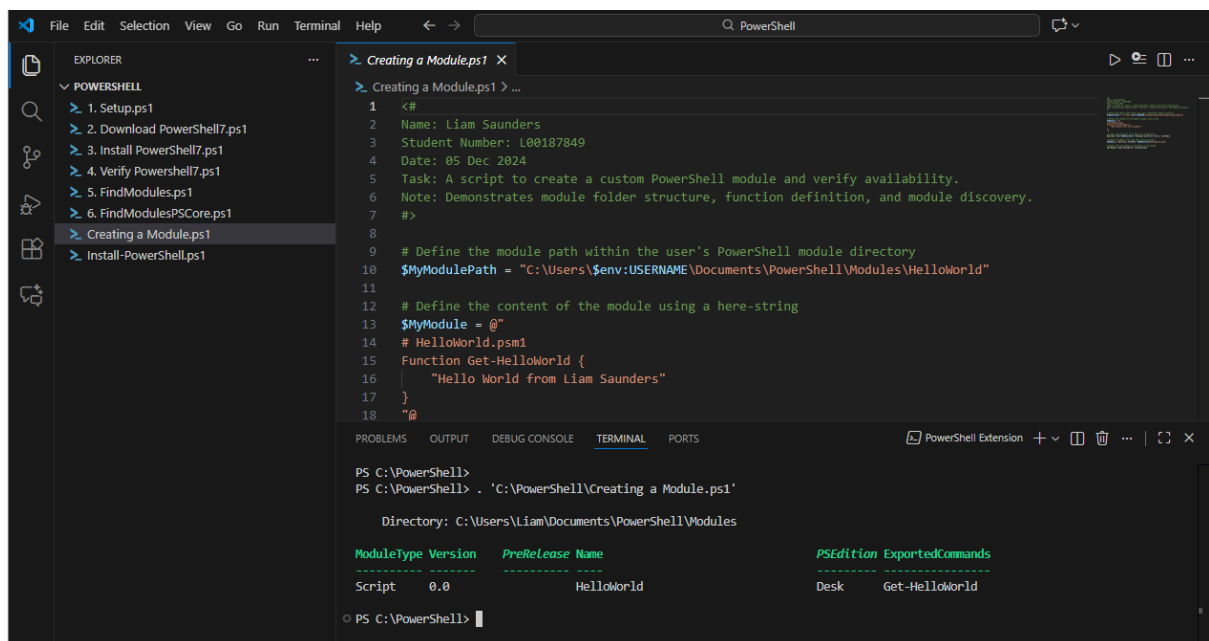
```
6. FindModulesPSCore.ps1
1 <#
2 Name: Liam Saunders
3 Student Number: L00187849
4 Date: 05 Dec 2024
5 Task: A script to find modules which support PowerShell Core. Includes an additional command to
6 Note: This command may take some time to run.
7 #>
8
9 # Find modules that explicitly support PowerShell Core
10 $PGSM = Find-Module -Name * -Tag 'PSEdition_Core'
11 "There are {0:N0} modules that support PowerShell Core." -f $PGSM.Count
12
13 # Optional command to list all available modules
14 # Get-Module -ListAvailable
15
```

PS C:\PowerShell> get-module -ListAvailable

Directory: C:\program files\powershell\7\Modules

ModuleType	Version	PreRelease Name	PSEdition	ExportedCommands
Manifest	7.0.0.0	CimCmdlets	Core	{Get-CimAssociatedInstance, Get-CimClass, Get-...
Manifest	1.2.5	Microsoft.PowerShell.Archive	Desk	{Compress-Archive, Expand-Archive}
Manifest	7.0.0.0	Microsoft.PowerShell.Diagnostics	Core	{Get-MinEvent, New-MinEvent, Get-Counter}
Manifest	7.0.0.0	Microsoft.PowerShell.Host	Core	{Start-Transcript, Stop-Transcript}
Manifest	7.0.0.0	Microsoft.PowerShell.Management	Core	{Add-Content, Clear-Content, Get-Clipboard, S...
Binary	1.1.1	Microsoft.PowerShell.PSResourceGet	Core, Desk	{Compress-PSResource, Find-PSResource, Get-In...
Manifest	7.0.0.0	Microsoft.PowerShell.Security	Core	{Get-Acl, Set-Acl, Get-PfxCertificate, Get-Cr...
Manifest	7.0.0.0	Microsoft.PowerShell.Utility	Core	{Export-Alias, Get-Alias, Import-Alias, New-A...
Manifest	7.0.0.0	Microsoft.WSMan.Management	Core	{Disable-WSManCredSSP, Enable-WSManCredSSP, G...
Script	1.4.8.1	PackageManagement	Desk	{Find-Package, Get-Package, Get-PackageProvid...
Script	2.2.5	PowerShellGet	Desk	{Find-Command, Find-DSRCResource, Find-Module...
Script	7.0.0.0	PSDiagnostics	Core	{Disable-PSTrace, Disable-PSWSManCombinedTrac...
Script	2.3.6	PSReadLine	Desk	{Get-PSReadLineKeyHandler, Set-PSReadLineKeyH...
Binary	2.0.3	ThreadJob	Desk	Start-ThreadJob

Figure 26: FindModulesPSCore.ps1 Output



```
Creating a Module.ps1
1 <#
2 Name: Liam Saunders
3 Student Number: L00187849
4 Date: 05 Dec 2024
5 Task: A script to create a custom PowerShell module and verify availability.
6 Note: Demonstrates module folder structure, function definition, and module discovery.
7 #>
8
9 # Define the module path within the user's PowerShell module directory
10 $MyModulePath = "C:\Users\$env:USERNAME\Documents\PowerShell\Modules\HelloWorld"
11
12 # Define the content of the module using a here-string
13 $MyModule = @"
14 # HelloWorld.psm1
15 Function Get-HelloWorld {
16     "Hello World from Liam Saunders"
17 }
18 "@
```

PS C:\PowerShell> . 'C:\PowerShell\Creating a Module.ps1'

Directory: C:\Users\Liam\Documents\PowerShell\Modules

ModuleType	Version	PreRelease Name	PSEdition	ExportedCommands
Script	0.0	HelloWorld	Desk	Get-HelloWorld

Figure 27: Creating a Module.ps1 Output

Laboratory Report

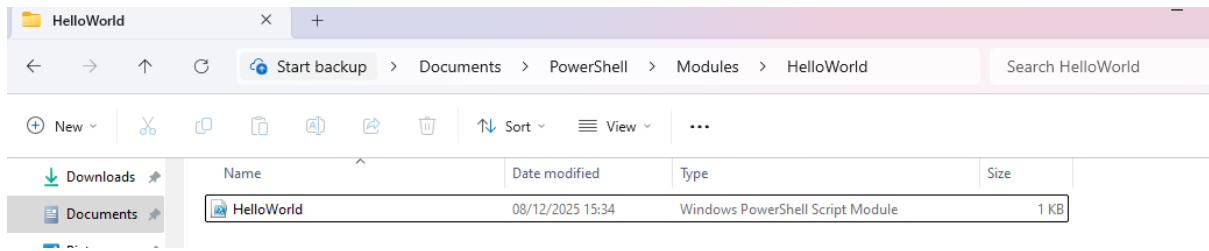


Figure 28: .psm1 file creation in Documents/PowerShell/Modules/HelloWorld Directory

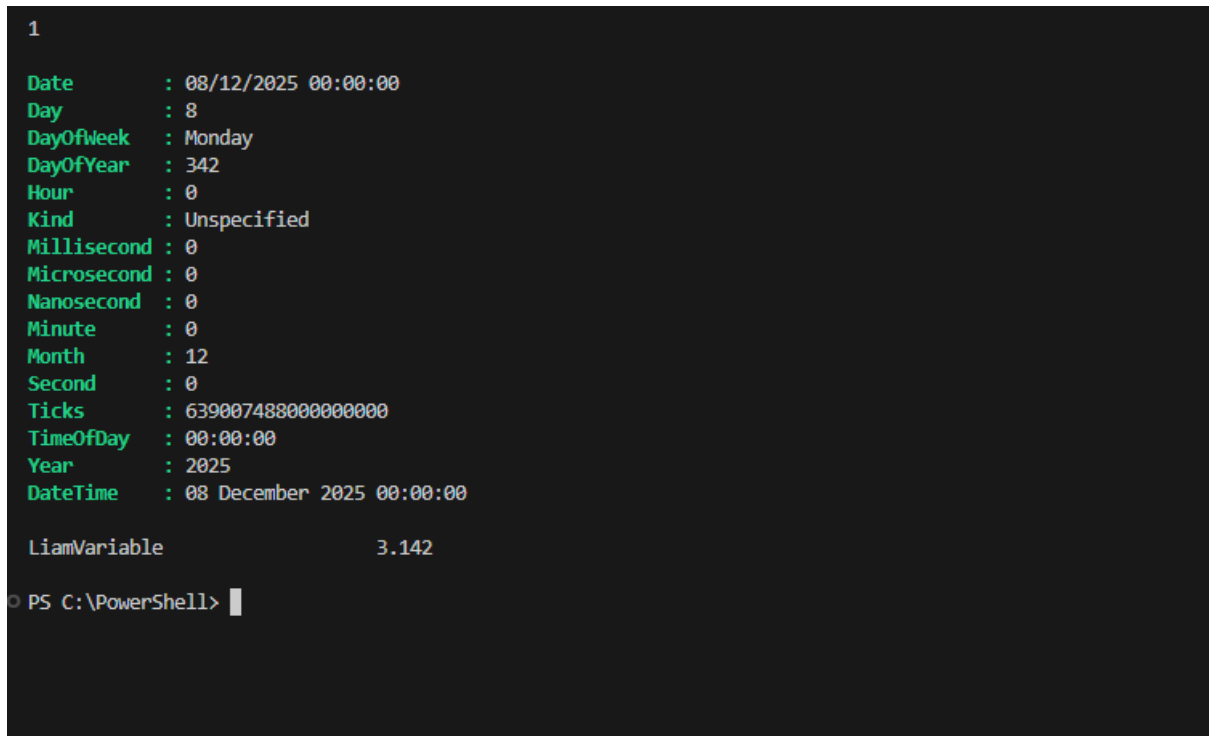


Figure 29: Variables.ps1 Output - Displays variable contents and type behaviour

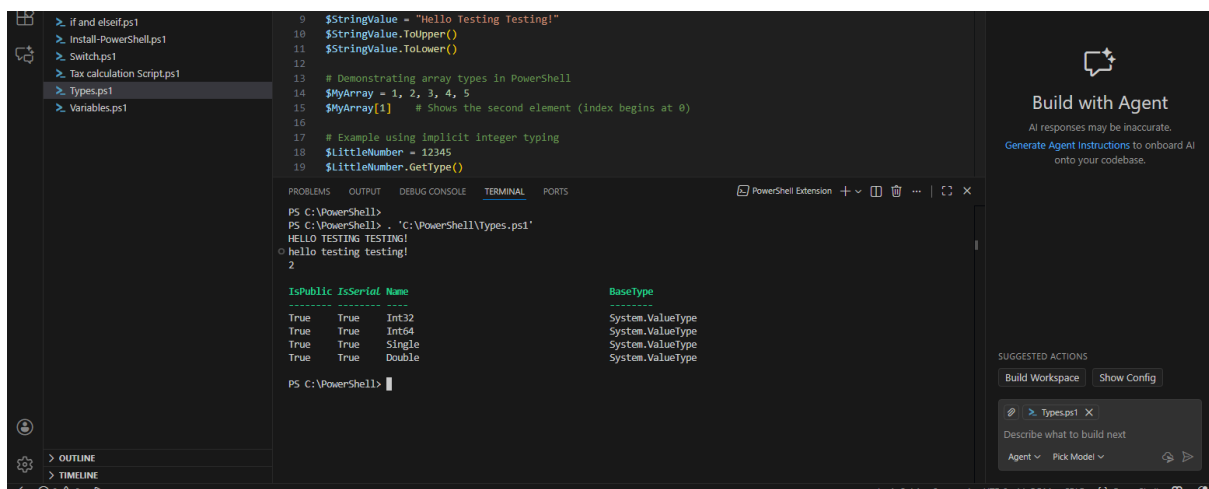
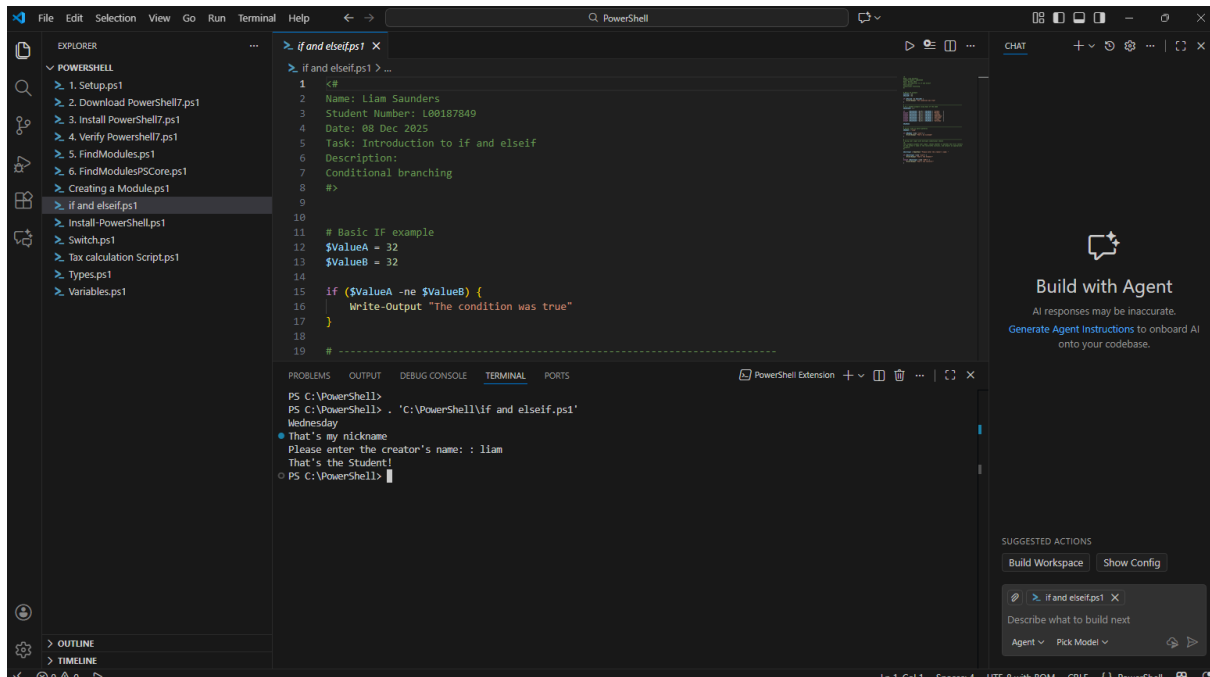


Figure 30: Types.ps1 Output

Laboratory Report



```
1 <#
2 Name: Liam Saunders
3 Student Number: L00187849
4 Date: 08 Dec 2025
5 Task: Introduction to if and elseif
6 Description:
7 Conditional branching
8 #>
9
10
11 # Basic IF example
12 $ValueA = 32
13 $ValueB = 32
14
15 if ($ValueA -ne $ValueB) {
16     Write-Output "The condition was true"
17 }
18
19 # -----
```

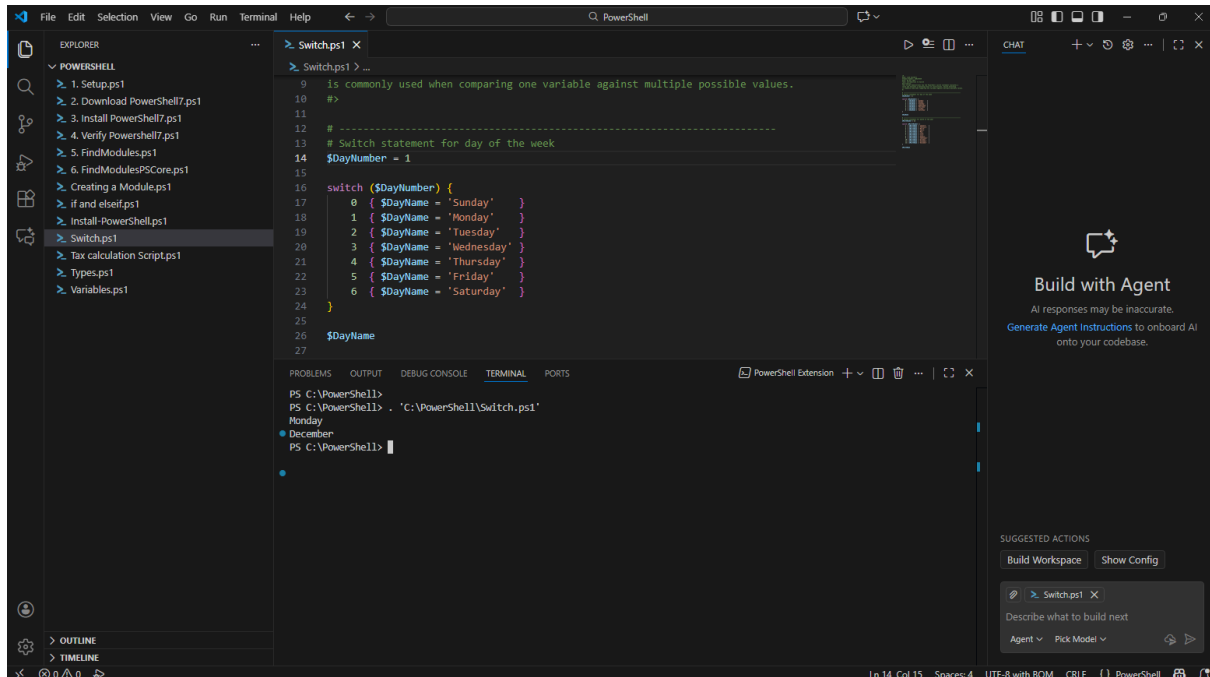
PS C:\PowerShell> . 'C:\PowerShell\if and elseif.ps1'

Wednesday

- That's my nickname
- Please enter the creator's name : liam
- That's the Student!

PS C:\PowerShell>

Figure 31: if and elseif.ps1 Output



```
9 is commonly used when comparing one variable against multiple possible values.
10 #>
11
12 # ----- statement for day of the week
13 $DayNumber = 1
14
15
16 switch ($DayNumber) {
17     0 { $DayName = 'Sunday' }
18     1 { $DayName = 'Monday' }
19     2 { $DayName = 'Tuesday' }
20     3 { $DayName = 'Wednesday' }
21     4 { $DayName = 'Thursday' }
22     5 { $DayName = 'Friday' }
23     6 { $DayName = 'Saturday' }
24 }
25
26 $DayName
27
```

PS C:\PowerShell> . 'C:\PowerShell\Switch.ps1'

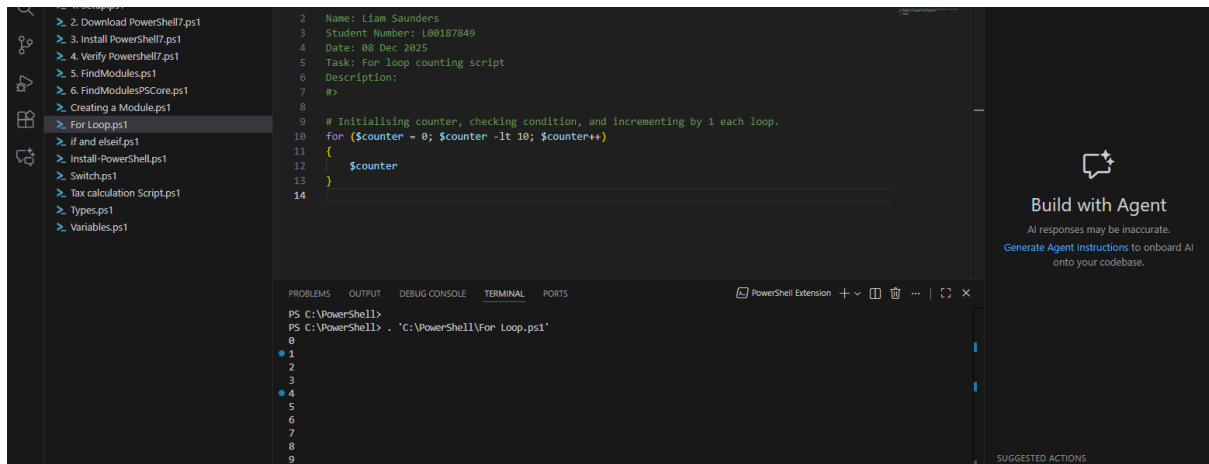
Monday

- December

PS C:\PowerShell>

Figure 32: Switch.ps1 Output

Laboratory Report

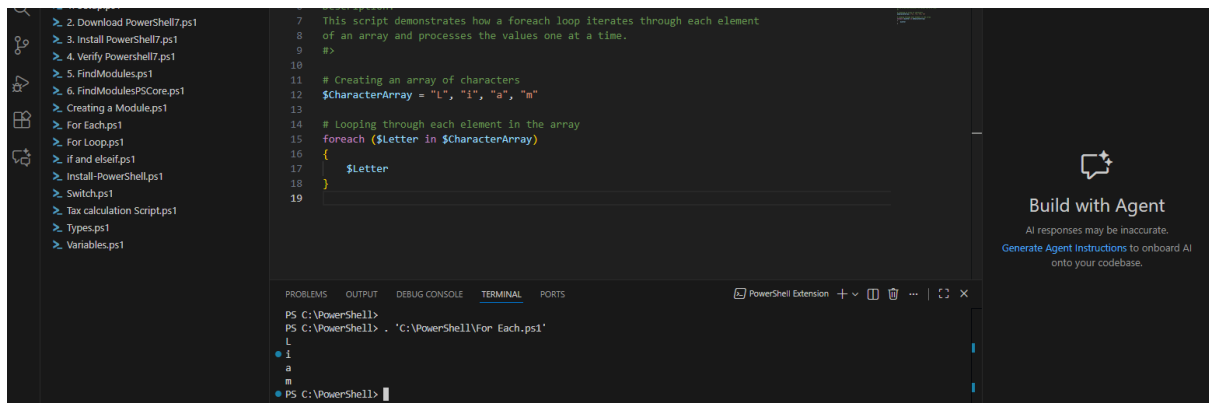


```
1 Name: Liam Saunders
2 Student Number: 100187849
3 Date: 08 Dec 2025
4 Task: For loop counting script
5 Description:
6 #>
7
8
9 # Initialising counter, checking condition, and incrementing by 1 each loop.
10 for ($counter = 0; $counter -lt 10; $counter++)
11 {
12     $counter
13 }
14
```

PS C:\PowerShell>
PS C:\PowerShell> . 'C:\PowerShell\For Loop.ps1'

0
1
2
3
4
5
6
7
8
9

Figure 33: For Loop.ps1 Output - Displays incrementing numeric values

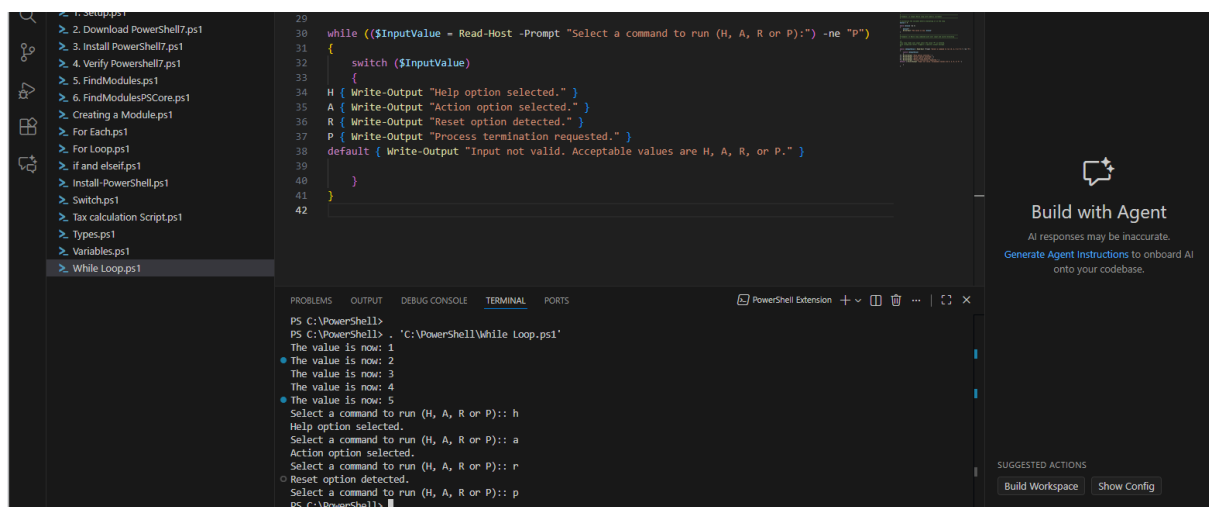


```
7 This script demonstrates how a foreach loop iterates through each element
8 of an array and processes the values one at a time.
9 #>
10
11 # Creating an array of characters
12 $CharacterArray = "l", "i", "a", "m"
13
14 # Looping through each element in the array
15 foreach ($Letter in $CharacterArray)
16 {
17     $Letter
18 }
19
```

PS C:\PowerShell>
PS C:\PowerShell> . 'C:\PowerShell\For Each.ps1'

l
i
a
m
PS C:\PowerShell>

Figure 34: For Each.ps1 Output - Prints each letter in sequence



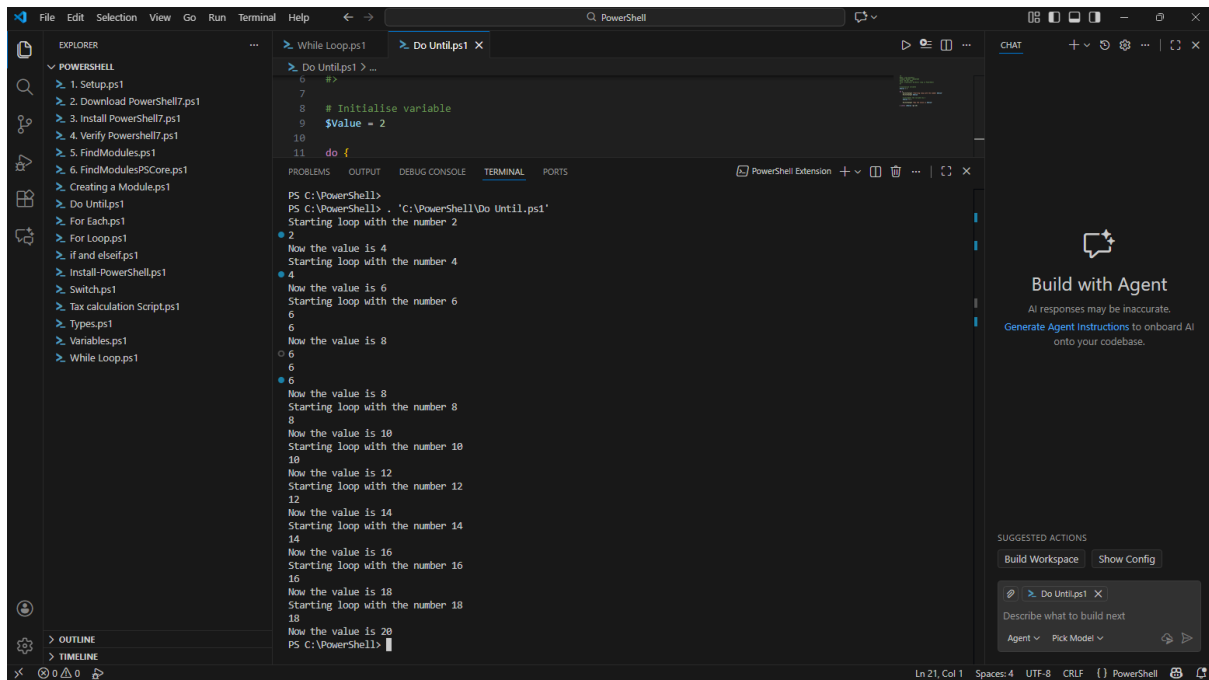
```
29 while (($InputValue -ne Read-Host -Prompt "Select a command to run (H, A, R or P):") -ne "P")
30 {
31     switch ($InputValue)
32     {
33         H { Write-Output "Help option selected." }
34         A { Write-Output "Action option selected." }
35         R { Write-Output "Reset option detected." }
36         P { Write-Output "Process termination requested." }
37         default { Write-Output "Input not valid. Acceptable values are H, A, R, or P." }
38     }
39 }
40
41
42
```

PS C:\PowerShell>
PS C:\PowerShell> . 'C:\PowerShell\While Loop.ps1'

The value is now: 1
The value is now: 2
The value is now: 3
The value is now: 4
The value is now: 5
Select a command to run (H, A, R or P):: h
Help option selected.
Select a command to run (H, A, R or P):: a
Action option selected.
Select a command to run (H, A, R or P):: r
Reset option detected.
Select a command to run (H, A, R or P):: p
PS C:\PowerShell>

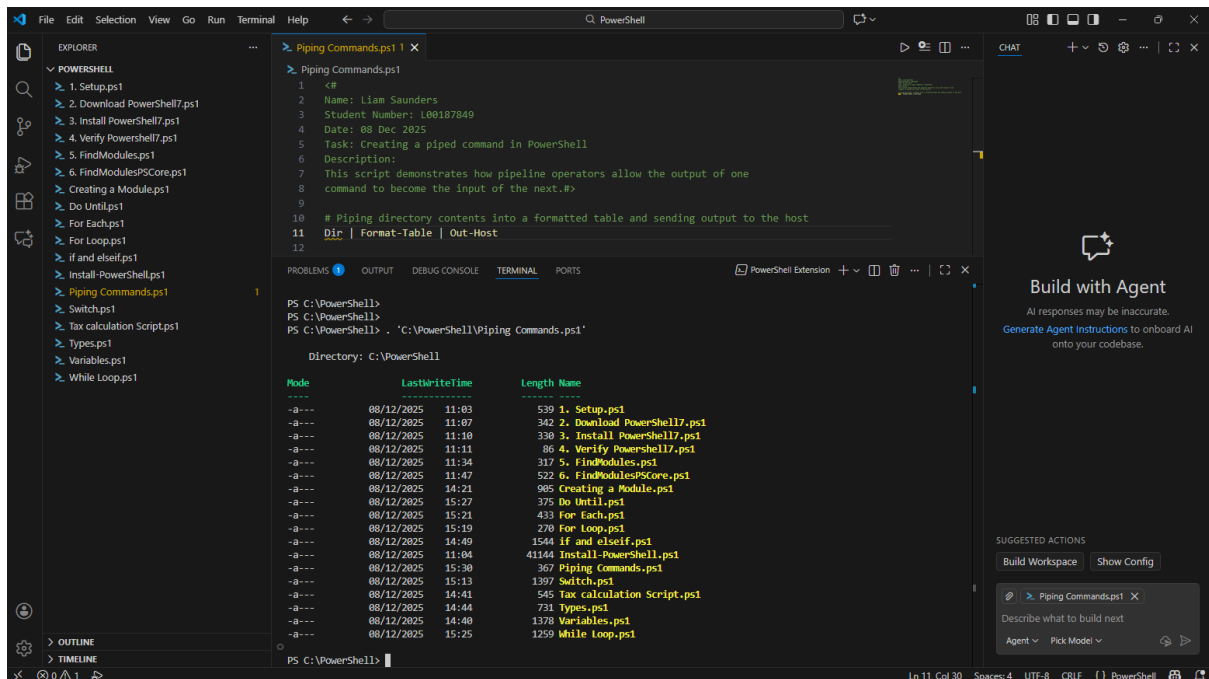
Figure 35: While Loop.ps1 Output - Repeated messages until stop condition

Laboratory Report



```
6 #>
7
8 # Initialise variable
9 $value = 2
10
11 do {
12     Now the value is 2
13     Starting loop with the number 2
14 } while ($value -lt 20)
15
16 Now the value is 4
17 Starting loop with the number 4
18
19 Now the value is 6
20 Starting loop with the number 6
21
22 Now the value is 8
23 Starting loop with the number 8
24
25 Now the value is 10
26 Starting loop with the number 10
27
28 Now the value is 12
29 Starting loop with the number 12
30
31 Now the value is 14
32 Starting loop with the number 14
33
34 Now the value is 16
35 Starting loop with the number 16
36
37 Now the value is 18
38 Starting loop with the number 18
39
40 Now the value is 20
41 PS C:\PowerShell>
```

Figure 36: Do Until.ps1 Output - Shows increasing values until limit reached



```
1 <&&
2 Name: Liam Saunders
3 Student Number: L08187849
4 Date: 08 Dec 2025
5 Task: Creating a piped command in PowerShell
6 Description:
7 This script demonstrates how pipeline operators allow the output of one
8 command to become the input of the next.
9
10 # Piping directory contents into a formatted table and sending output to the host
11 Dir | Format-Table | Out-Host
12
```

Mode	LastWriteTime	Length	Name
-a---	08/12/2025 11:03	539	1. Setup.ps1
-a---	08/12/2025 11:07	342	2. Download PowerShell7.ps1
-a---	08/12/2025 11:10	330	3. Install PowerShell7.ps1
-a---	08/12/2025 11:11	86	4. Verify PowerShell7.ps1
-a---	08/12/2025 11:34	317	5. FindModules.ps1
-a---	08/12/2025 11:47	522	6. FindModulesPSCore.ps1
-a---	08/12/2025 14:21	985	Creating a Module.ps1
-a---	08/12/2025 15:27	375	Do Until.ps1
-a---	08/12/2025 15:21	433	For Each.ps1
-a---	08/12/2025 15:19	270	For Loop.ps1
-a---	08/12/2025 14:49	1544	If and Elseif.ps1
-a---	08/12/2025 11:04	41144	Install-PowerShell.ps1
-a---	08/12/2025 15:30	367	Piping Commands.ps1
-a---	08/12/2025 15:13	1307	Switch.ps1
-a---	08/12/2025 14:41	545	Tax calculation Script.ps1
-a---	08/12/2025 14:44	731	Types.ps1
-a---	08/12/2025 14:40	1378	Variables.ps1
-a---	08/12/2025 15:25	1259	While Loop.ps1

Figure 37: Piping Commands.ps1 Output - Formatted directory listing displayed